

BaoClaw README

BaoClaw v1.0.0

The AI coding agent that remembers, evolves, and follows you everywhere.

[English](#) · [中文](#)

What is BaoClaw?

BaoClaw is an open-source AI coding agent with a Rust core engine, persistent memory, cross-device session sharing, a cron scheduler, and a self-evolution loop. It runs as a single global daemon on your machine, managing multiple project sessions simultaneously. Your terminal, Telegram, and WhatsApp all connect to this one daemon — each routed to the correct project session by working directory.

Unlike agents that forget everything when you close the window, BaoClaw builds up knowledge about you and your projects over time. The more you use it, the better it gets.

Key Features

Persistent Memory

- **Project-level memory** — each project directory gets its own `memory.jsonl`
- **Global memory** — cross-project facts, preferences, and decisions in `~/.baoclaw/`
- **Long-term recall** — memories are injected into the system prompt automatically
- **Manual control** — `/memory add`, `/memory list`, `/memory delete`

Multi-Client, Global Daemon

- **One daemon, all projects** — a single daemon process manages sessions for all your project directories
- **Per-project sessions** — each cwd gets its own session with independent history and memory
- **Cross-device** — start a task on your laptop terminal, continue on Telegram from your phone
- **Real-time streaming** — all clients see tool calls and responses as they happen
- **No conflicts** — two CLI terminals in different directories use different sessions, no interference
- **Session persistence** — conversations survive daemon restarts, auto-resumed per project

Self-Evolution Engine

Inspired by [Hermes Agent](#)'s learning loop: - **Trajectory recording** — every interaction is logged with tools used, outcomes, and timing - **Skill auto-generation** — complex successful tasks are extracted as reusable skill candidates - **Self-evaluation nudge** — every 15 tasks, the agent reflects on patterns and creates/improves skills - **User ratings** — rate interactions as good/bad to build preference data - **RLHF data export** — export trajectories as JSONL for DPO/RLHF fine-tuning of smaller models - **Personal evolution** — skills and trajectories are cross-project (`~/.baoclaw/evolution/`) - **Evolve tool** — agent can autonomously create, improve, and promote skills

Cron Scheduler

- **Periodic tasks** — schedule prompts to run automatically inside the daemon
- **Flexible schedules** — every 30m, every 2h, daily 09:00, weekly mon 09:00
- **Result broadcast** — cron results pushed to all connected clients (CLI + Telegram)
- **Persistent** — jobs saved in ~/.baocl原因/cron.json, survive daemon restarts
- **Full agent power** — each job runs with complete tool access

Document Q&A

- **Upload files** — PDF, DOCX, and images via Telegram or CLI (@file.pdf)
- **Route A** — client-side text extraction (mammoth for DOCX, pdf-parse for PDF)
- **Route B** — native API document blocks (PDF sent directly to Claude/OpenAI)
- **Image understanding** — photos analyzed via multimodal API (both Anthropic and OpenAI compatible)
- **Tab completion** — @ triggers file path completion in CLI

Project-Scoped Everything

- **/cd command** — switch working directory at runtime, like changing projects
- **Auto-scaffold** — .baocl原因/ directory with config files created automatically
- **Session per project** — each directory maps to its own persistent session file
- **Auto-resume** — reconnecting to a project automatically restores conversation history
- **Project instructions** — BAOCRAW.md loaded into system prompt per project
- **Memory isolation** — each project has its own memory store

15+ Built-in Tools

Tool	Description
Bash	Shell commands (respects project cwd)
FileRead / FileWrite / FileEdit	File operations with path validation
Grep / Glob	Code search and file discovery
WebSearch	Brave Search API with retry on rate limits
WebFetch	Fetch and parse web pages
Memory	Long-term memory management
Agent	Sub-agent for parallel tasks
Evolve	Self-improvement: create/improve skills, export training data
Todo	Task list management
Notebook	Jupyter notebook editing
ProjectNote	Project-level notes
ToolSearch	Search across all registered tools

Extensible

- **MCP support** — connect external MCP servers for additional tools
- **Skills** — markdown-based skill files loaded into system prompt (personal + project scope)
- **Plugins** — directory-based plugin system with tools, skills, and MCP configs
- **200+ LLM models** — Anthropic native + any OpenAI-compatible API (OpenRouter, Ollama, vLLM, etc.)

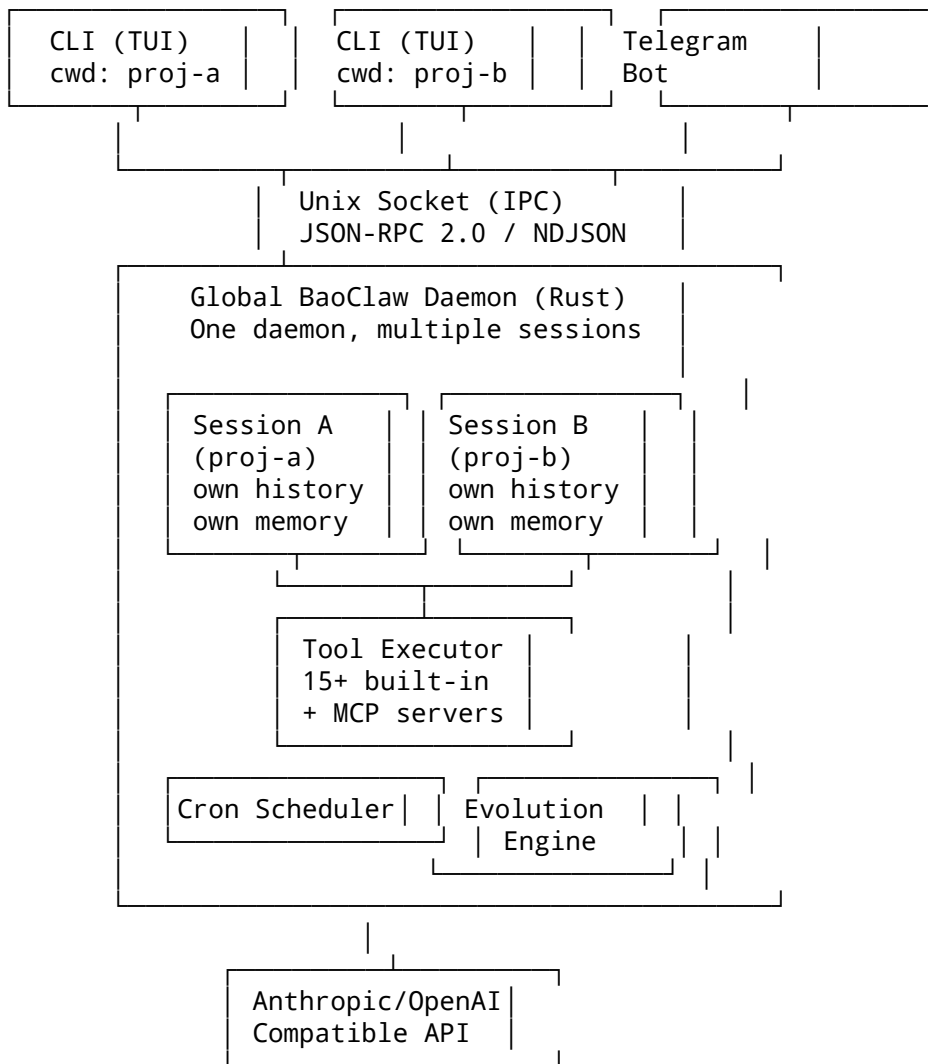
Model Fallback

- **Automatic retry** — rate-limited requests retry with exponential backoff
- **Fallback chain** — configure multiple models; if one is rate-limited, fall back to the next
- **Transparent** — CLI shows model switches in real-time

Keyboard Shortcuts

- **Ctrl+C** during task → abort current task
- **Ctrl+C** when idle → hint to press again or /quit
- **Ctrl+C × 2** → disconnect from daemon
- **Tab** → autocomplete commands and file paths

Architecture



Key design: **one global daemon process manages all projects**. Each project directory gets its own session with independent conversation history and memory. Multiple CLI terminals and Telegram can connect simultaneously — each routed to the correct project session by its working directory.

Internals: How the Engine Works

This section describes the five core mechanisms that make BaoClaw tick: **Memory**, **Context**, **Evolution**, **System Prompt**, and **Model Fallback**. All are implemented in the Rust core engine (baoclaw-

1. 🧠 Memory Mechanism

BaoClaw has two complementary memory layers: **Long-Term Memory** (cross-session facts/preferences) and **Session Memory** (rolling summary within a conversation).

Long-Term Memory (memory.jsonl)

Aspect	Detail
Scope	Two levels: global (~/.baoclaw/memory.jsonl) and project (<project>/.baoclaw/memory.jsonl)
Categories	fact (user told me X), preference (user prefers Y), decision (we decided Z)
Storage	Append-only JSONL, one JSON object per line
Injection	Loaded at daemon startup → build_prompt_fragment() → appended to system prompt
Management	/memory add, /memory list, /memory delete, /memory clear

When the daemon starts, MemoryStore::load() reads both files, and build_prompt_fragment() generates a formatted block that becomes part of the append_system_prompt injected into every conversation turn.

Session Memory (session_memory.rs)

A per-session rolling summary that persists across the lifetime of a session — like meeting notes that get refined over time.

Aspect	Detail
Storage	~/.baoclaw/sessions/{session_id}.memory.md
First update	Triggers at 4 messages (if summary is empty)
Refresh interval	Every 10 messages after the last update
Thread safety	std::sync::Mutex — safe to share via Arc<SessionMemory>
Persistence	Written to disk on every update() call

How it's used: 1. **Free compaction** — session_memory_compact() uses the existing summary to replace old messages without any API call (keeps last 10 messages) 2. **Dynamic reminder** — injected into <system-reminder> in the user message each turn alongside git status 3. **Session resume** — when reconnecting, the old summary seeds the new session and triggers immediate compaction if > 50 messages

2. 📐 Context Mechanism

BaoClaw manages a 200K-token context window with a multi-layer compaction strategy and accurate token counting.

Token Counting (token_counter.rs)

Aspect	Detail
Context window	200,000 tokens (default)
Auto-compact threshold	70% = 140,000 tokens
Tokenizer	c1100k_base (GPT-4 tokenizer, ~5-10% over-count for Claude)
Counting strategy	Calibrated from API response <code>usage.input_tokens</code> → anchored baseline + tiktoken delta
Baseline persistence	<code>~/.baoclaw/sessions/{id}.baseline.json</code> — restores calibration after restart

Budget levels (for 200K window):

Level	Threshold	Action
Normal	< 140K	Continue normally
Compact	≥ 140K (70%)	Pre-emptive compaction triggered
Warning	≥ 147K	Log warning
Blocking	≥ 164K	MUST compact before next API call

4-Level Compaction Hierarchy

Compaction is tried from cheapest to most expensive:

Level 1: <code>micro_compact</code> (FREE, every turn) - Clears <code>tool_result</code> content > 500 chars AND > 60 min old - Skips last 4 messages (current turn) - Replacement text: "[Old tool result cleared – N chars]"	
Level 2: <code>session_memory_compact</code> (FREE, no API call) - Uses existing <code>SessionMemory</code> rolling summary - Keeps last 10 messages, prepends <code>CompactBoundary</code> - Triggered when budget = Compact/Blocking	
Level 3: <code>compact_messages</code> (1 API call, cache-safe) - Keeps last 10 messages, summarizes older ones via API - Cache-safe forking: reuses system prompt + old messages as API messages → cache prefix reuse on provider side - Summary input truncated to 60,000 chars (~15K tokens) - Circuit breaker: skipped after 3 consecutive failures	
Level 4: <code>reactive_compact</code> (FREE, last resort) - Groups messages into turns, drops oldest 20% - Guard: won't drop if ≤ 4 messages or ≤ 2 turns	
Level 5: <code>inline compact</code> (on <code>context_overflow</code> error) - Triggered when API returns "<code>model_context_window_exceeded</code>" - Keeps last 4 messages, summarizes old via inline API call - Retries the query with compacted context	

System Prompt Architecture

The system prompt is split into **static** (cached) and **dynamic** (per-turn) parts to maximize API prompt caching:

Static part (build_system_prompt()) — tagged with cache_control: ephemeral: 1. Core system prompt (or custom override) 2. Working directory + “show full content” instruction 3. Project instructions from BAOCLAW.md 4. Project rules from .baoclaw/rules/*.md (path-filtered against recent files) 5. Append system prompt = **skills** + **long-term memory** + **evolution prompt**

Dynamic part (build_dynamic_reminder()) — injected into the **last user message** as <system-reminder>: 1. Git status (branch, staged/modified/untracked files) 2. Session memory (rolling summary)

This split ensures the cached system prompt prefix stays stable across turns — only the dynamic reminder changes.

Session Resume Flow — Summary-First Three-Tier Strategy

- 1. find_latest_session_for_cwd(cwd)
 - FNV-1a hash of cwd → scan ~/.baoclaw/sessions/ for matching .jsonl
- 2. TranscriptWriter::load(session_id) → read all entries
- 3. SessionMemory::load(session_id) → check .memory.md for existing summary
- 4. Three-tier loading (10 min → < 5 sec):
 - Tier 1 (best): summary exists → load summary + last 200 entries only
 - Tier 2 (small): entries ≤ 400, no summary → safe to rebuild all
 - Tier 3 (fallback): large session, no summary → last 200 entries + warning
- 5. engine.set_messages(messages)
- 6. engine.load_token_baseline(session_id) // restore calibrated count
- 7. engine.seed_session_memory(&old_summary) // carry forward to new session

Background summary generation ensures Tier 1 is always available: - First update at 6 messages, then every 10 messages (tokio::spawn, non-blocking) - Session close heuristic fallback if background never ran - Pre-query compact safety: >500 messages → session_memory_compact (free) or tail-trim (no API call)

3. ↻ Evolution Mechanism

The self-evolution engine learns from every interaction to create and improve reusable skills.

File Layout

- ~/.baoclaw/evolution/
 - ├─ trajectories.jsonl # Every interaction record (append-only)
 - ├─ session_summaries.jsonl # Structured summary per session close
 - ├─ skill_stats.json # Per-skill usage tracking
 - ├─ pending_review.json # Cross-session review → next session's prompt
 - ├─ pending_eval.json # Self-evaluation nudge (one-shot, consumed)
 - ├─ candidates/
 - └─ {skill-name}.json # Auto-extracted skill candidates

Key Thresholds

Constant	Value	Purpose
SKILL_CREATION_THRESHOLD	3 tool calls	Min complexity to auto-extract a skill candidate
SELF_EVAL_INTERVAL	15 tasks	Trigger self-evaluation nudge
Review trigger	≥ 2 turns	Only generate pending_review.json if session had ≥ 2 turns
Candidate name max	60 chars	Slugified from user prompt

Constant	Value	Purpose
Topic truncation	200 chars	Per topic in session summary

Evolution Lifecycle

During interaction (record_trajectory):

Every user interaction

- Append trajectory to trajectories.jsonl
- Increment task_count
- IF tool_count $\geq$ 3 AND outcome = Completed:
 - Auto-extract SkillCandidate → save to candidates/{name}.json
- Every 15 tasks → write pending_eval.json (one-shot nudge)

Session close (on_session_close) — pure Rust, no LLM call:

Last client disconnects

- Extract from message history:
 - user_topics, tool_usage frequency, errors, skills_used
- Write session_summaries.jsonl
- IF turn_count $\geq$ 2:
 - Write pending_review.json (for next session's system prompt)

System prompt injection (build_prompt_fragment):

Start of new session

- Check pending_review.json from previous session
 - Generate "Last Session Review" with self-improvement nudges
- Check pending_eval.json
 - Generate "Self-Evaluation Nudge"
- List pending skill candidates
- All injected into append_system_prompt → system prompt layer 5

Skill promotion (promote_skill):

Candidate approved → Move from candidates/ to ~/.baoclaw/skills/{name}.md
 Remove candidate file
 Skill loaded in all future sessions

Training export (export_training_data):

Read all trajectories → Create preference pairs

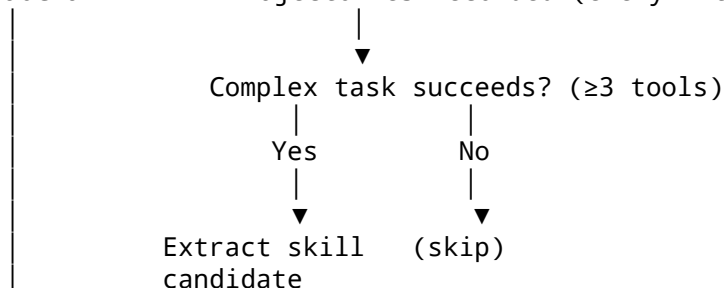
Each pair: { prompt, response, rating: chosen/rejected/neutral }

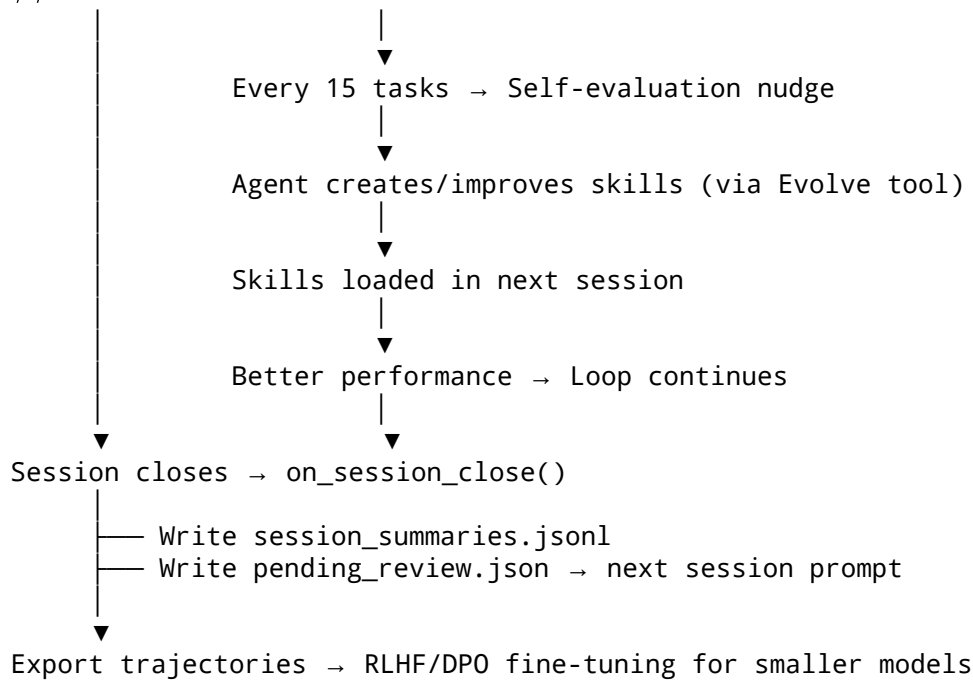
Output: ~/.baoclaw/evolution/training_export.jsonl

Suitable for DPO/RLHF fine-tuning

Full Evolution Loop

Use BaoClaw —→ Trajectories recorded (every interaction)





4. 📁 System Prompt Construction

The system prompt is assembled in 5 ordered layers. The order matters for API prompt caching — stable layers come first:

Layer 1: Core System Prompt • Default: "You are a helpful AI coding assistant." • Override: custom_system_prompt in config • Includes: "show full content" instruction • Cache: cache_control = ephemeral	
Layer 2: Working Directory • Current cwd path • Instructs agent to output full file content	
Layer 3: Project Instructions • From BAOCLAW.md (project root or .baoclaw/) • Loaded once, cached across turns • Also loads .baoclaw/rules/*.md (path-filtered)	
Layer 4: Append System Prompt • Skills (personal ~/.baoclaw/skills/ + project) • Long-term memory (facts, preferences, decisions) • Evolution prompt (pending reviews, skill candidates)	
Dynamic <system-reminder> (in last USER message) • NOT in system prompt – preserves cache stability • Git status (branch, changed files) • Session memory (rolling summary)	

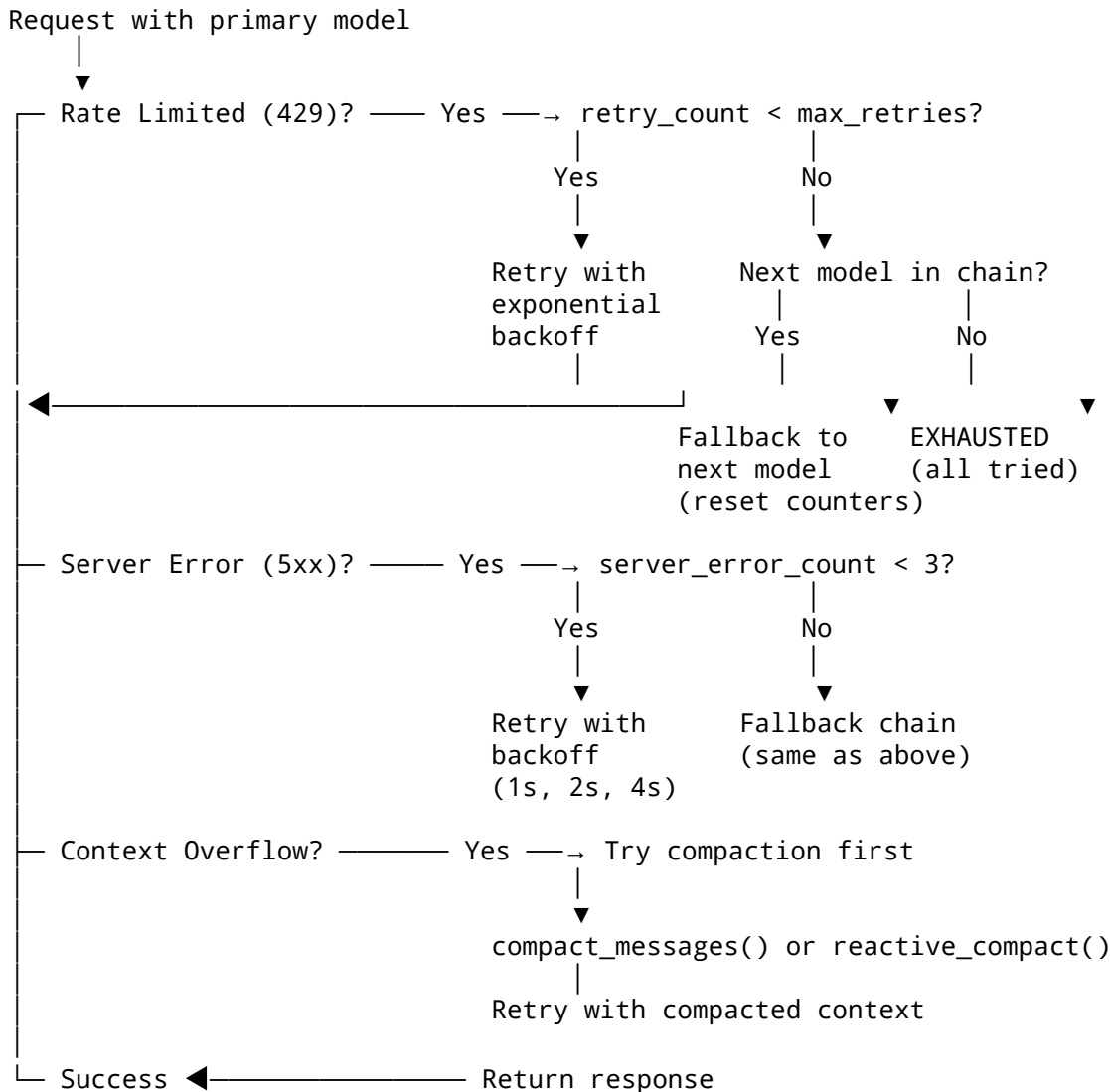
Why this split? The static layers (1-4) are tagged with cache_control: ephemeral so the API provider can cache the prefix. Only the dynamic <system-reminder> changes every turn — and it's injected into the user message, not the system prompt, so the system prompt cache stays warm.

How skills & memory are loaded: 1. At daemon startup: `load_skills_for_prompt(cwd)` → discovers all skill .md files 2. At daemon startup: `MemoryStore::load()` → reads global + project memory.jsonl 3. Combined via `build_append_prompt()` → becomes Layer 4 4. Evolution engine's `build_prompt_fragment()` adds pending reviews and skill candidates 5. All of this is computed once at startup and reused across turns

5. 🔄 Model Fallback Mechanism

When the primary model is unavailable or rate-limited, BaoClaw automatically falls back through a configurable chain of models.

Fallback Chain



Configuration

```

{
  "model": "claude-sonnet-4-20250514",
  "fallback_models": ["claude-3-5-haiku-20241022"],
  "max_retries_per_model": 2
}

```

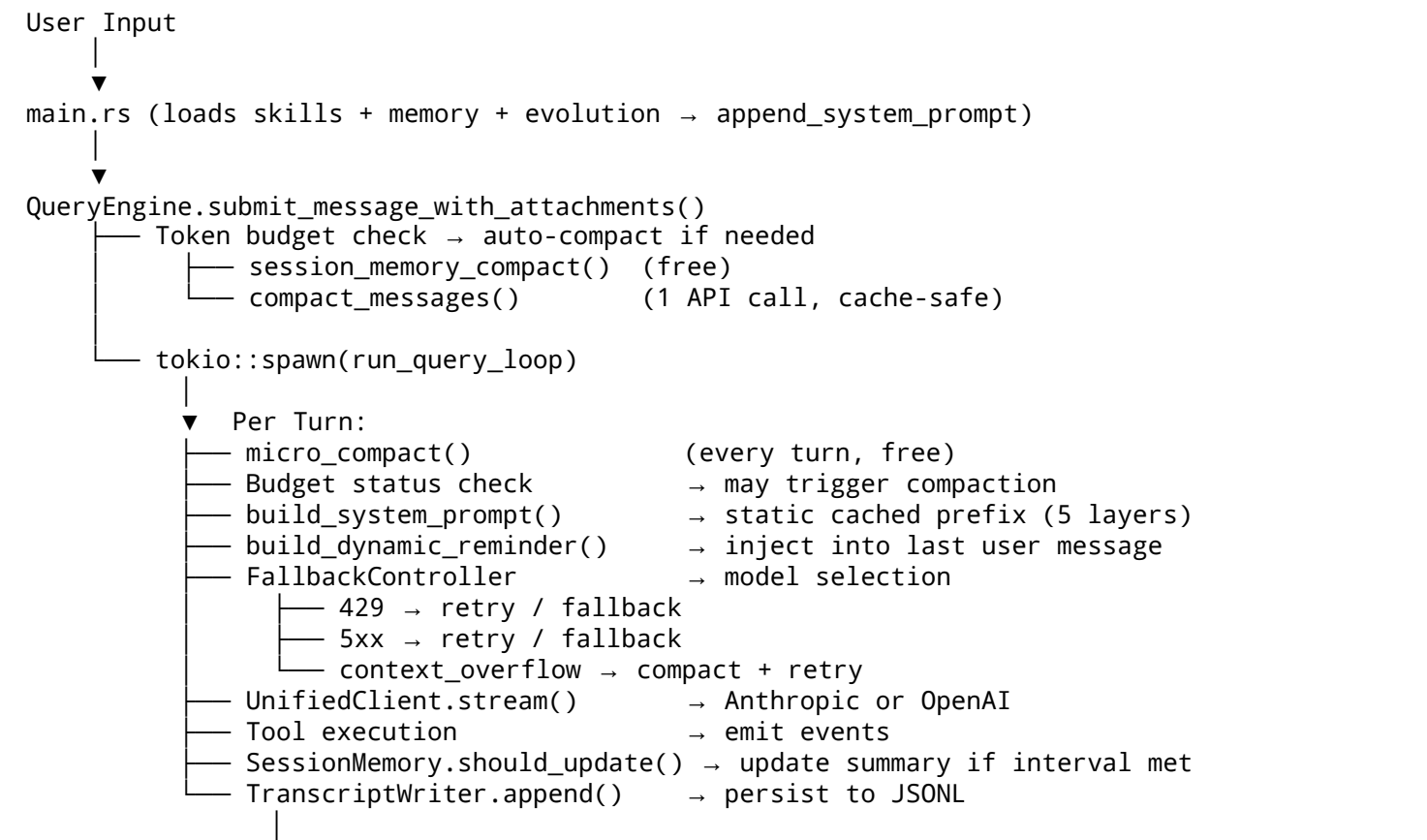
Parameter	Default	Description
model	claude-sonnet-4-20250514	Primary model (tried first every time)
fallback_models	[]	Ordered list of fallback models
max_retries_per_model	2	Retries per model before falling back
Server error max retries	3	Built-in limit for 5xx errors

Error Recovery Strategies

Error Type	Strategy	Parameters
IPC disconnect	Restart process	Full daemon restart
State sync failed	Full state sync	Re-sync from scratch
API rate limited (429)	Retry with backoff	3 attempts, 1s initial delay
API server error (5xx)	Retry with backoff	3 attempts, exponential (1s → 2s → 4s)
API auth error	Fatal	Cannot recover automatically
API bad request (context overflow)	Auto-compact	Compact → retry
MCP disconnect	Retry	5 attempts, 2s initial delay
Tool timeout	Fatal	Report to user

Key behaviors: - The fallback controller **resets** to the primary model for each new query (cross-turn stateless) - **Exponential backoff** prevents hammering a rate-limited endpoint - **Circuit breaker:** After 3 consecutive compaction failures, auto-compaction is disabled to avoid wasting API calls - **5-minute timeout** per API call — on timeout, the user message is removed to keep history clean

Data Flow Summary



```

▼ Session Close:
EvolutionEngine.on_session_close()
├── Write session_summaries.jsonl
├── Write pending_review.json (→ next session)
└── Skill extraction if applicable

```

Installation

Prerequisites

- **Rust** (1.75+) — rustup.rs
- **Node.js** (18+) — nodejs.org
- An LLM API key (Anthropic, OpenRouter, or any OpenAI-compatible provider)

Linux / macOS

```

git clone https://github.com/baohx/BaoClaw.git
cd BaoClaw
./install.sh

```

The installer builds the Rust core, installs Node.js dependencies, and creates the baoclaw launcher in `~/.local/bin/`.

Windows (WSL2)

BaoClaw requires a Unix environment. On Windows, use WSL2:

```

# Install WSL2 if not already installed
wsl --install

# Inside WSL2
git clone https://github.com/baohx/BaoClaw.git
cd BaoClaw
./install.sh

```

Manual Setup

```

# 1. Build Rust core
cd baoclaw-core
cargo build --release
cd ..

# 2. Install CLI dependencies
cd ts-ipc
npm install
cd ..

# 3. Set your API key
export ANTHROPIC_API_KEY=sk-ant-...
# Or for OpenAI-compatible:
export ANTHROPIC_API_KEY=your-key
export ANTHROPIC_BASE_URL=https://your-provider.com/v1

# 4. Run
npx --prefix ts-ipc tsx ts-ipc/cli.ts

```

Configuration Reference

Directory Structure

```
~/ .baoclaw/                                     # User-level (global, cross-project)
├── config.json                                  # Main configuration
├── memory.jsonl                                # Global memories (fallback)
├── cron.json                                   # Scheduled tasks
├── sessions/                                  # Session transcripts (per-project)
│   └── {cwd_hash}-{uuid}.jsonl
├── skills/                                    # Personal skills (cross-project)
│   └── my-skill.md
├── plugins/                                   # User-level plugins
│   └── my-plugin/
│       ├── skills/
│       └── mcp.json
├── mcp.json                                   # User-level MCP servers
├── mcp-auth/                                  # MCP OAuth tokens
├── models/                                   # Local model files (whisper etc.)
│   └── ggml-base.bin
├── telemetry/                                # Telemetry events (local only)
├── evolution/                                # Self-evolution data
│   ├── trajectories.jsonl                    # Interaction history for RLHF
│   ├── candidates/                          # Auto-extracted skill candidates
│   └── training_export.jsonl                 # Exported training data
├── telegram-gateway.pid                       # Telegram gateway PID file
└── telegram-gateway.log                       # Telegram gateway log

<project>/ .baoclaw/                             # Project-level
├── BAOCLAW.md                                # Project instructions → system prompt
├── mcp.json                                   # Project MCP servers
├── mcp.local.json                             # Local MCP overrides (gitignored)
├── memory.jsonl                              # Project-level memories
├── skills/                                   # Project-specific skills
├── plugins/                                   # Project-level plugins
├── backups/                                  # File backups before edits
└── todo.json                                 # Project todo list
```

~/ .baoclaw/config.json — Main Configuration

```
{
  "model": "claude-sonnet-4-20250514",
  "fallback_models": ["claude-3-5-haiku-20241022"],
  "max_retries_per_model": 2,
  "api_type": "anthropic",
  "openai_base_url": null,
  "telegram": {
    "token": "123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11",
    "allowedChatIds": [12345678]
  }
}
```

Field	Type	Default	Description
model	string	claude-sonnet-4-20250514	Primary LLM model
fallback_models	string[]	[]	Models to try when primary is rate-limited
max_retries_per_model	number	2	Retries before falling back to next model
api_type	string	"anthropic"	"anthropic" or "openai"

Field	Type	Default	Description
openai_base_url	string?	null	Base URL for OpenAI-compatible API
telegram.token	string	—	Telegram bot token from @BotFather
telegram.allowedChatIds	number[]	[]	Allowed chat IDs (empty = allow all)

Environment variable overrides: - ANTHROPIC_API_KEY — API key (required) - ANTHROPIC_MODEL — overrides model field - ANTHROPIC_BASE_URL — overrides openai_base_url - BRAVE_SEARCH_API_KEY — for WebSearch tool

OpenAI-compatible example:

```
{
  "model": "deepseek-chat",
  "api_type": "openai",
  "openai_base_url": "https://api.deepseek.com/v1"
}
```

<project>/**.baoclaw/BAOCLAW.md — Project Instructions**

Injected into the system prompt for every conversation in this project. Write anything the agent should know about your project.

My Project

This is a Python web app using FastAPI + SQLAlchemy.

Conventions

- Use type hints everywhere
- Tests go in tests/ directory
- Use pytest for testing
- Database migrations with alembic

Important Files

- src/main.py — app entry point
- src/models/ — SQLAlchemy models
- src/api/ — FastAPI routes

Also works as BAOCLAW.md in the project root (.baoclaw/BAOCLAW.md takes priority).

mcp.json — MCP Server Configuration

Works at both user level (~/.baoclaw/mcp.json) and project level (<project>/**.baoclaw/mcp.json**). Project-level overrides user-level.

```
{
  "mcpServers": {
    "sqlite": {
      "command": "uvx",
      "args": ["mcp-server-sqlite", "--db-path", "./data.db"],
      "env": {},
      "disabled": false
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
```

```

    "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_..."
  }
}
}
}

```

<project>/.**baoclaw**/mcp.local.json — same format, for local overrides that should be gitignored.

~/.**baoclaw**/cron.json — Scheduled Tasks

Managed via `/cron` command. Do not edit manually while daemon is running.

```

[
  {
    "id": "a1b2c3d4",
    "name": "Daily git summary",
    "prompt": "Summarize yesterday's git commits in this project",
    "schedule": "daily 09:00",
    "cwd": "/home/user/my-project",
    "enabled": true,
    "created_at": "2026-04-14T10:00:00Z",
    "last_run": "2026-04-14T09:00:15Z",
    "last_result": "3 commits: fixed login bug, added tests..."
  }
]

```

Schedule formats: every 30m, every 2h, daily 09:00, weekly mon 09:00

~/.**baoclaw**/skills/ and <project>/.**baoclaw**/skills/ — Skills

Markdown files loaded into the system prompt. User-level skills apply to all projects; project-level skills apply only to that project.

```

---
description: Code review checklist
created_by: evolution
version: 2
---

```

Code Review

When asked to review code:

1. Check for security issues (SQL injection, XSS, etc.)
2. Check error handling (are errors caught and logged?)
3. Check naming conventions
4. Suggest performance improvements
5. Output as a checklist with ✓/✗

Skills can be created manually or auto-generated by the Evolution Engine.

~/.**baoclaw**/evolution/trajectories.jsonl — Interaction Trajectories

Auto-recorded. Each line is a JSON object:

```

{
  "id": "a1b2c3d4",
  "timestamp": "2026-04-14T10:30:00Z",

```

```
"cwd": "/home/user/project",
"user_prompt": "Fix the login bug",
"assistant_actions": [
  {"tool_name": "Grep", "input_summary": "search for login", "output_summary": "found in
auth.py", "is_error": false},
  {"tool_name": "FileEdit", "input_summary": "fix auth.py line 42", "output_summary":
"edited", "is_error": false}
],
"outcome": {"Completed": {"final_text_preview": "Fixed the login bug by..."}},
"tool_count": 2,
"duration_ms": 15000,
"user_rating": "Good"
}
```

Export for RLHF fine-tuning: ask the agent to export training data or use the Evolve tool.

CLI Commands

Command	Description
/projects	Project management: list, switch, new, desc
/tools	List registered tools
/mcp	List MCP servers
/skills	List loaded skills
/plugins	List installed plugins
/model [name]	Show or switch model
/think	Toggle extended thinking mode
/compact	Compress conversation context
/memory	Long-term memory: list, add, delete, clear
/cron	Scheduled tasks: add, list, remove, toggle
/diff	Git diff summary
/commit <msg>	Stage all and commit
/git	Git status (branch, changes)
/task	Background tasks: run, list, status, stop
/voice	Voice input (requires whisper.cpp)
/telegram	Manage Telegram gateway: start, stop, status
/telemetry	Toggle telemetry on/off
@file.pdf	Attach file for Q&A (PDF, DOCX, images)
/abort	Cancel current request (or press Ctrl+C)
/clear	Clear screen
/help	Show all commands
/quit	Disconnect (daemon keeps running)
/shutdown	Stop the daemon process

Telegram Commands

All CLI commands are also available in Telegram:

Command	Description
/tools /skills /mcp /plugins	List resources
/model [name]	Show or switch model
/think	Toggle extended thinking
/compact	Compress context
/memory	Manage memories
/cron	Manage scheduled tasks
/projects	项目管理: list, switch, new, desc
/task	Manage background tasks
/diff /commit /git	Git operations
/abort	Cancel current task
/status	Gateway status
/help	Show all commands
📎 Upload file	Send PDF/DOCX/image for Q&A

Telegram Setup

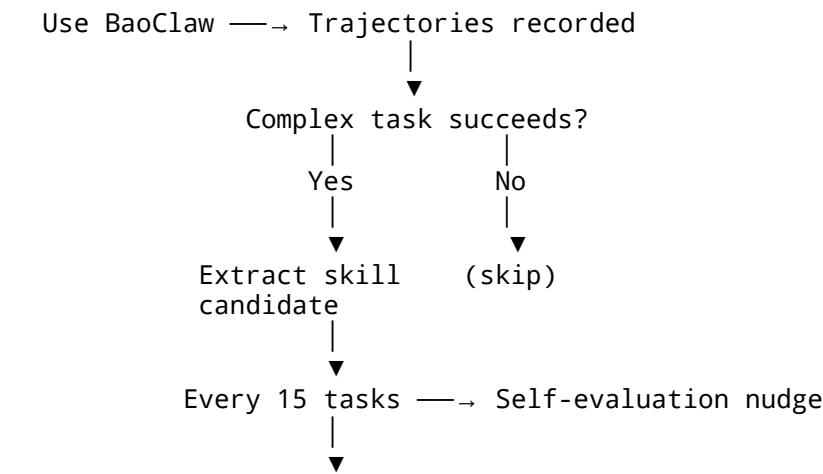
1. Create a bot via [[@BotFather](https://t.me/BotFather)](https://t.me/BotFather)
 2. Add token to ~/.baoclaw/config.json
 3. Start from CLI: `/telegram start`
- Upload documents and images directly in Telegram chat — the bot extracts text and sends it to the AI.

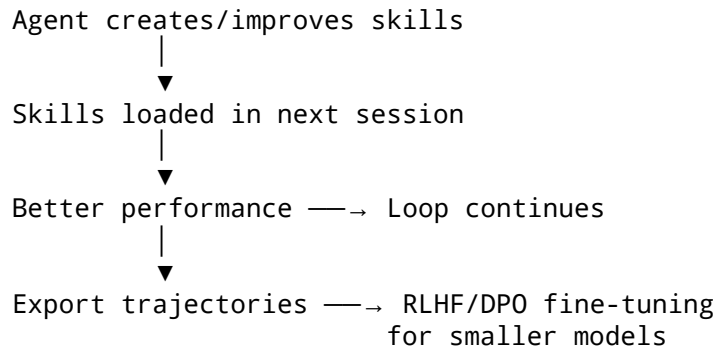
Cron Examples

```
/cron add "Daily git summary" "daily 09:00" Summarize yesterday's git commits
/cron add "Dep check" "weekly mon 10:00" Check for dependency security updates
/cron add "Evolution review" "every 2h" Review pending skill candidates and improve
/cron list
/cron toggle abc123
/cron remove abc123
```

Results are pushed to all connected clients (CLI shows 📬 notification, Telegram receives a message).

Self-Evolution: How It Works





Training Data Export

```
# Inside BaoClaw, ask the agent:
> Export training data for fine-tuning
```

```
# Or use the Evolve tool directly:
# The agent calls Evolve(operation: "export_training")
# Output: ~/.baoclaw/evolution/training_export.jsonl
```

Each trajectory contains: prompt, tool actions, outcome, user rating (good/bad/neutral). Rated trajectories can be used as preference pairs for DPO training.

License

MIT

🐾 BaoClaw — 会记忆、会进化、跨设备的 AI 编程助手

BaoClaw 是一个开源 AI 编程 Agent，基于 Rust 核心引擎，具备持久记忆、跨设备会话共享、定时任务和自我进化能力。它以守护进程方式运行，同时连接终端、Telegram 和 WhatsApp，所有客户端共享同一个对话上下文。

和那些关掉窗口就失忆的 Agent 不同，BaoClaw 会随着使用不断积累对你和你项目的了解。用得越多，越好用。

核心特性

🧠 持久记忆

- 项目级记忆 — 每个项目目录独立的 memory.jsonl
- 全局记忆 — 跨项目的个人偏好和决策
- 自动注入 — 记忆自动加载到系统提示词中
- 手动管理 — /memory add、/memory list、/memory delete

🏠 全局守护进程，多客户端

- 一个守护进程管所有项目 — 单个 daemon 进程管理所有项目目录的会话
- 项目级会话 — 每个工作目录有独立的会话历史和记忆
- 跨设备 — 在电脑终端开始任务，用手机 Telegram 继续
- 实时流式输出 — 所有客户端同步看到工具调用和响应
- 无冲突 — 两个终端在不同目录工作，使用不同会话，互不干扰
- 会话持久化 — 对话在守护进程重启后自动恢复，按项目目录绑定

🔧 自我进化引擎

参考 [Hermes Agent](#) 的学习循环： - 轨迹记录 — 每次交互自动记录工具调用、结果和耗时 - Skill 自动生成 — 复杂的成功任务自动提取为可复用的 skill 候选 - 自我评估 — 每 15 个任务触发反思，创建或改进 skill - 用户评价 — 对交互评分 (good/bad)，构建偏好数据 - RLHF 数据导出 — 导出轨迹数据用于小模型的 DPO/RLHF 微调 - 个人级进化 — skill 和轨迹跨项目积累 (~/.baoclaw/evolution/) - Evolve 工具 — Agent 可自主创建、改进和提升 skill

🕒 定时任务

- 周期执行 — 在守护进程内自动运行预设的提示词
- 灵活调度 — every 30m、every 2h、daily 09:00、weekly mon 09:00
- 结果推送 — 定时任务结果推送到所有连接的客户端 (终端 + Telegram)
- 持久化 — 任务保存在 ~/.baoclaw/cron.json，守护进程重启后自动恢复
- 完整能力 — 每个任务都拥有完整的 Agent 工具访问权限

📖 文档问答

- 上传文件 — 通过 Telegram 或终端 (@file.pdf) 上传 PDF、DOCX、图片
- 文本提取 — DOCX 用 mammoth，PDF 用 pdf-parse
- 原生文档 — PDF 可直接发送给 Claude API
- 图片理解 — 支持 Anthropic 和 OpenAI 兼容 API 的多模态
- Tab 补全 — 终端中输入 @ 后按 Tab 自动补全文件路径

🗂️ 项目级隔离

- /cd 命令 — 运行时切换工作目录，相当于切换项目
- 自动初始化 — 新目录自动创建 .baoclaw/ 配置骨架
- 项目绑定会话 — 每个目录对应独立的持久化会话文件
- 自动恢复 — 重连时自动恢复项目的对话历史
- 项目指令 — BAOCLAW.md 按项目加载到系统提示词
- 记忆隔离 — 每个项目有独立的记忆存储

🔧 15+ 内置工具

Bash、文件读写编辑、Grep、Glob、Web 搜索、Web 抓取、记忆管理、子 Agent、自我进化、Todo、Notebook 编辑、项目笔记、工具搜索等。

可扩展

- MCP 协议 — 连接外部 MCP 服务器获取更多工具
- Skills — Markdown 格式的技能文件（个人级 + 项目级）
- 插件系统 — 目录式插件，包含工具、技能和 MCP 配置
- 200+ 模型 — Anthropic 原生 + 任意 OpenAI 兼容 API

模型降级

- 自动重试 — 限流时指数退避重试
- 降级链 — 配置多个模型，限流时自动切换
- 透明提示 — 终端实时显示模型切换

快捷键

- Ctrl+C（任务中）→ 中止当前任务
- Ctrl+C（空闲时）→ 提示再按一次退出
- Ctrl+C × 2 → 断开连接
- Tab → 自动补全命令和文件路径

内部机制：引擎工作原理

本节描述 BaoClaw 的五个核心机制：**记忆**、**上下文**、**进化**、**系统提示词**和**模型退回**。全部在 Rust 核心引擎（baoclaw-core/src/engine/）中实现。

1. 记忆机制

BaoClaw 有两个互补的记忆层：**长期记忆**（跨会话的事实/偏好）和**会话记忆**（对话内的滚动摘要）。

长期记忆 (memory.jsonl)

方面	细节
作用域	两级：全局（~/.baoclaw/memory.jsonl）和项目级（<项目>/.baoclaw/memory.jsonl）
分类	fact（用户告知的事实）、preference（用户偏好）、decision（决策记录）
存储	追加写入 JSONL，每行一个 JSON 对象
注入方式	守护进程启动时加载 → build_prompt_fragment() → 追加到系统提示词
管理命令	/memory add、/memory list、/memory delete、/memory clear

守护进程启动时，MemoryStore::load() 读取两个文件，build_prompt_fragment() 生成格式化文本块，成为每次对话都注入的 append_system_prompt 的一部分。

会话记忆 (session_memory.rs)

每个会话的滚动摘要，在会话生命周期内持续精炼 —— 就像不断完善的会议纪要。

方面	细节
存储位置	~/.baoclaw/sessions/{session_id}.memory.md
首次更新	4 条消息时触发（如果摘要是空的）
刷新间隔	每隔 10 条消息更新一次
线程安全	std::sync::Mutex — 可通过 Arc<SessionMemory> 安全共享
持久化	每次 update() 调用都立即写入磁盘

用途：1. **免费压缩** — session_memory_compact() 用已有摘要替换旧消息，无需 API 调用（保留最近 10 条）2. **动态提醒** — 每轮注入到用户消息的 <system-reminder> 中，与 git 状态并列3. **会话恢复** — 重连时，旧摘要种子到新会话，超过 50 条消息时立即触发压缩

2. 上下文机制

BaoClaw 管理 200K token 的上下文窗口，采用多层压缩策略和精确的 token 计数。

Token 计数 (token_counter.rs)

方面	细节
上下文窗口	200,000 tokens（默认）
自动压缩阈值	70% = 140,000 tokens
分词器	cl100k_base（GPT-4 分词器，对 Claude 约多算 5-10%）
计数策略	从 API 响应的 usage.input_tokens 校准 → 锚定基线 + tiktoken 增量
基线持久化	~/.baoclaw/sessions/{id}.baseline.json — 重启后恢复校准值

预算等级（200K 窗口）：

等级	阈值	行为
Normal	< 140K	正常运行
Compact	≥ 140K (70%)	触发预防性压缩
Warning	≥ 147K	记录警告日志
Blocking	≥ 164K	必须在下一次 API 调用前压缩

5 级压缩层次

从代价最低到最高依次尝试：

第 1 级：micro_compact （免费，每轮执行）	
• 清除 > 500 字符 且 > 60 分钟的 tool_result 内容	
• 跳过最近 4 条消息（当前轮次）	
• 替换文本："[Old tool result cleared – N chars]"	
第 2 级：session_memory_compact （免费，无需 API）	
• 使用已有的 SessionMemory 滚动摘要	
• 保留最近 10 条消息，前置 CompactBoundary	
• 预算状态为 Compact/Blocking 时触发	
第 3 级：compact_messages （1 次 API 调用，缓存安全）	
• 保留最近 10 条消息，通过 API 摘要旧消息	
• 缓存安全分叉：复用系统提示词 + 旧消息作为 API 消息	
→ 在提供商侧复用缓存前缀	
• 摘要输入截断到 60,000 字符（约 15K tokens）	
• 熔断器：连续失败 3 次后跳过	
第 4 级：reactive_compact （免费，最后手段）	
• 按轮次分组消息，丢弃最早的 20%	
• 保护：≤ 4 条消息或 ≤ 2 轮时不丢弃	
第 5 级：inline compact （上下文溢出错误时触发）	
• API 返回 "model_context_window_exceeded" 时触发	
• 保留最近 4 条消息，通过内联 API 调用摘要旧消息	
• 用压缩后的上下文重试查询	

会话恢复流程 —— 摘要优先三层策略

1. find_latest_session_for_cwd(cwd)
- 对 cwd 做 FNV-1a 哈希 → 扫描 ~/.baoclaw/sessions/ 匹配的 .jsonl
2. TranscriptWriter::load(session_id) → 读取所有条目
3. SessionMemory::load(session_id) → 检查 .memory.md 是否有预写摘要
4. 三层加载策略（10 分钟 → < 5 秒）：
- Tier 1（最佳）：有摘要 → 只加载摘要 + 最近 200 条 entries
- Tier 2（小会话）：entries ≤ 400，无摘要 → 安全 rebuild 全部
- Tier 3（兜底）：大会话无摘要 → 只取最后 200 条 + 警告消息
5. engine.set_messages(messages)
6. engine.load_token_baseline(session_id) // 恢复校准计数
7. engine.seed_session_memory(&old_summary) // 继承摘要到新 session

后台摘要生成确保 Tier 1 始终可用： - 首次在 6 条消息后更新，之后每 10 条消息更新一次
(tokio::spawn, 非阻塞) - Session 关闭时 heuristic 兜底（如果后台更新从未触发） - Pre-query compact
安全策略：>500 条消息 → session_memory_compact（免费）或 tail-trim（无 API 调用）

3. 进化机制

自我进化引擎从每次交互中学习，创建和改进可复用的技能。

文件布局

```
~/baoclaw/evolution/
├── trajectories.jsonl          # 每次交互记录（追加写入）
├── session_summaries.jsonl    # 每次会话关闭的结构化摘要
├── skill_stats.json           # 每个技能的使用追踪
├── pending_review.json        # 跨会话审查 → 下次会话的提示词
├── pending_eval.json          # 自我评估提醒（一次性消费）
└── candidates/
    └── {skill-name}.json      # 自动提取的技能候选
```

关键阈值

常量	值	用途
SKILL_CREATION_THRESHOLD	3 次工具调用	自动提取技能候选的最低复杂度
SELF_EVAL_INTERVAL	15 个任务	触发自我评估提醒
审查触发条件	≥ 2 轮	只有 ≥ 2 轮的会话才生成 pending_review.json
候选名称最大长度	60 字符	从用户提示词 slug 化
话题截断	200 字符	会话摘要中每个话题

进化生命周期

交互期间 (record_trajectory):

```
每次用户交互
├── 追加轨迹到 trajectories.jsonl
├── 递增 task_count
├── 如果 tool_count ≥ 3 且结果 = Completed:
│   └── → 自动提取 SkillCandidate → 保存到 candidates/{name}.json
└── 每 15 个任务 → 写入 pending_eval.json（一次性提醒）
```

会话关闭 (on_session_close) — 纯 Rust，无 LLM 调用：

```
最后一个客户端断开
├── 从消息历史提取：
│   └── 用户话题、工具使用频率、错误、已加载技能
├── 写入 session_summaries.jsonl
└── 如果 turn_count ≥ 2:
    └── 写入 pending_review.json（给下次会话的提示词）
```

系统提示词注入 (build_prompt_fragment):

新会话开始

- └─ 检查上次会话的 pending_review.json
 - └─ → 生成 "Last Session Review" 及自我改进建议
- └─ 检查 pending_eval.json
 - └─ → 生成 "Self-Evaluation Nudge"
- └─ 列出待处理的技能候选
- └─ 全部注入到 append_system_prompt → 系统提示词第 5 层

技能提升 (promote_skill):

- 候选被批准 → 从 candidates/ 移到 ~/.baoclaw/skills/{name}.md
 - 删除候选文件
 - 技能在所有后续会话中加载

训练数据导出 (export_training_data):

- 读取所有轨迹 → 创建偏好对
 - 每对: { prompt, response, rating: chosen/rejected/neutral }
 - 输出: ~/.baoclaw/evolution/training_export.jsonl
 - 适用于 DPO/RLHF 微调

4. 系统提示词构建

系统提示词由 5 个有序层组装。顺序对 API 提示词缓存至关重要 —— 稳定的层排在前面:

第 1 层: 核心系统提示词	
• 默认: "You are a helpful AI coding assistant." • 可覆盖: config 中的 custom_system_prompt • 包含: "显示完整内容"指令 • 缓存: cache_control = ephemeral	
第 2 层: 工作目录	
• 当前 cwd 路径 • 指示 Agent 输出完整文件内容	
第 3 层: 项目指令	
• 来自 BAOCLAW.md (项目根目录或 .baoclaw/) • 加载一次, 跨轮次缓存 • 同时加载 .baoclaw/rules/*.md (按路径过滤)	
第 4 层: 追加系统提示词	
• 技能 (个人级 ~/.baoclaw/skills/ + 项目级) • 长期记忆 (事实、偏好、决策) • 进化提示词 (待处理审查、技能候选)	
动态 <system-reminder> (在最后一用户消息中)	
• 不在系统提示词内 – 保持缓存稳定性 • Git 状态 (分支、修改文件) • 会话记忆 (滚动摘要)	

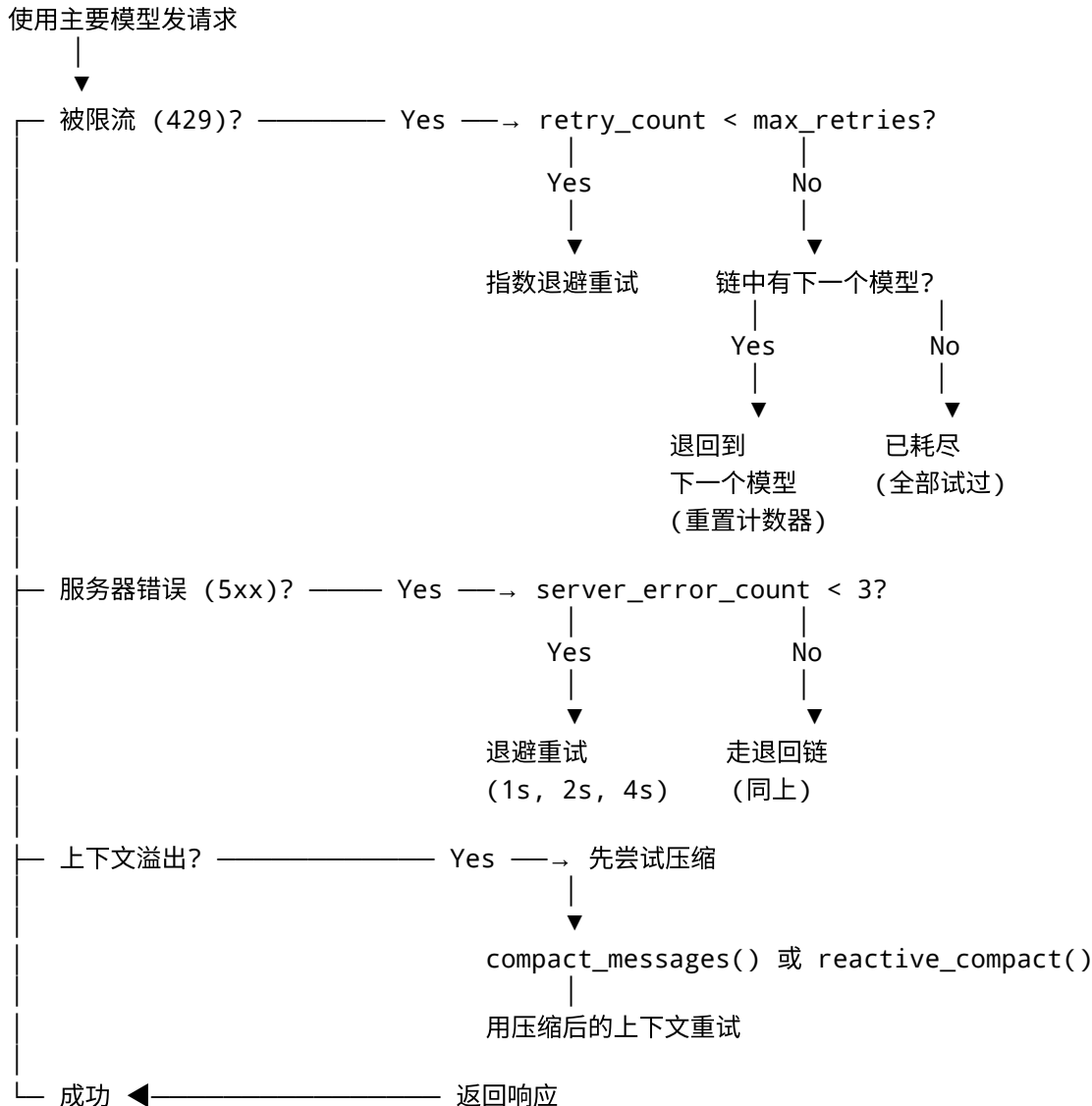
为什么这样拆分？ 静态层（1-4）标记了 `cache_control: ephemeral`，API 提供商可以缓存前缀。只有动态的 `<system-reminder>` 每轮变化——它被注入到用户消息而非系统提示词中，因此系统提示词缓存保持有效。

技能和记忆的加载流程： 1. 守护进程启动时: `load_skills_for_prompt(cwd)` → 发现所有技能 `.md` 文件
2. 守护进程启动时: `MemoryStore::load()` → 读取全局 + 项目 `memory.jsonl`
3. 通过 `build_append_prompt()` 合并 → 成为第 4 层
4. 进化引擎的 `build_prompt_fragment()` 添加待处理审查和技能候选
5. 所有这些在启动时计算一次，跨轮次复用

5. 🔄 模型退回机制

当主要模型不可用或被限时时，BaoClaw 自动退回到可配置的模型链中的下一个模型。

退回链



配置


```
{
  "model": "claude-sonnet-4-20250514",
  "fallback_models": ["claude-3-5-haiku-20241022"],
  "max_retries_per_model": 2
}
```

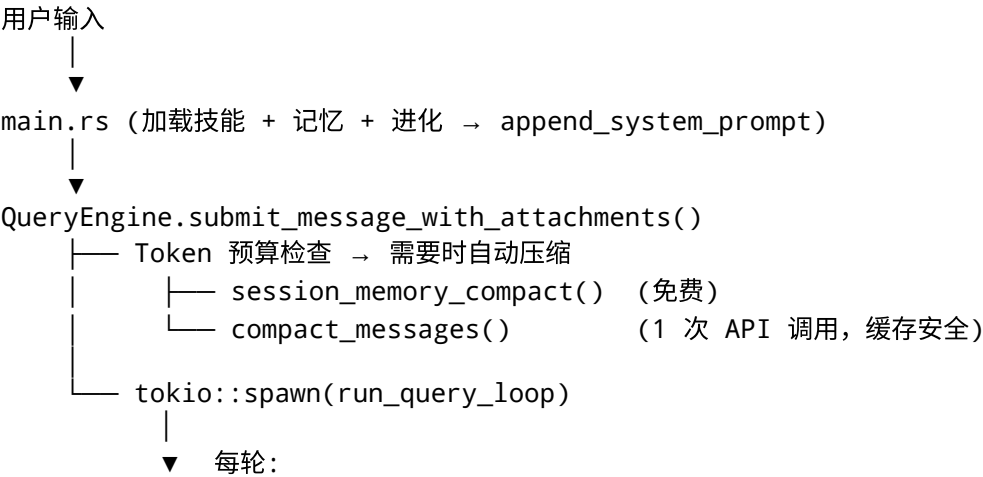
参数	默认值	说明
model	claude-sonnet-4-20250514	主要模型（每次优先尝试）
fallback_models	[]	有序的退回模型列表
max_retries_per_model	2	每个模型退回前的重试次数
服务器错误最大重试	3	5xx 错误的内置限制

错误恢复策略

错误类型	策略	参数
IPC 断连	重启进程	完整守护进程重启
状态同步失败	全量同步	从头重新同步
API 限流 (429)	退避重试	3 次尝试，初始延迟 1s
API 服务器错误 (5xx)	退避重试	3 次尝试，指数退避 (1s → 2s → 4s)
API 认证错误	致命	无法自动恢复
API 请求错误（上下文溢出）	自动压缩	压缩 → 重试
MCP 断连	重试	5 次尝试，初始延迟 2s
工具超时	致命	报告给用户

关键行为： - 退回控制器对每个新查询**重置**到主要模型（跨轮次无状态） - **指数退避**防止持续冲击限流端点 - **熔断器**：连续 3 次压缩失败后，禁用自动压缩以避免浪费 API 调用 - 每次 API 调用 **5 分钟超时** — 超时移除用户消息以保持历史记录干净

数据流总览



```

├─ micro_compact()           (每轮, 免费)
├─ 预算状态检查             → 可能触发压缩
├─ build_system_prompt()    → 静态缓存前缀 (5 层)
├─ build_dynamic_reminder() → 注入到最后一条用户消息
├─ FallbackController       → 模型选择
│   └─ 429 → 重试 / 退回
│   └─ 5xx → 重试 / 退回
│   └─ 上下文溢出 → 压缩 + 重试
├─ UnifiedClient.stream()   → Anthropic 或 OpenAI
├─ 工具执行                 → 发出事件
├─ SessionMemory.should_update() → 间隔到达时更新摘要
└─ TranscriptWriter.append() → 持久化到 JSONL
    |
    ▼ 会话关闭:
    EvolutionEngine.on_session_close()
    └─ 写入 session_summaries.jsonl
    └─ 写入 pending_review.json (→ 下次会话)
    └─ 适当时提取技能

```

安装

前置条件

- Rust (1.75+) — rustup.rs
- Node.js (18+) — nodejs.org
- LLM API Key (Anthropic、OpenRouter 或任意 OpenAI 兼容服务)

Linux / macOS

```

git clone https://github.com/baohx/BaoClaw.git
cd BaoClaw
./install.sh

```

Windows (WSL2)

```

wsl --install
# 在 WSL2 中
git clone https://github.com/baohx/BaoClaw.git
cd BaoClaw
./install.sh

```

使用

```

export ANTHROPIC_API_KEY=sk-ant-...
baoclaw

```

OpenAI 兼容模式:

```

export ANTHROPIC_API_KEY=your-key
export ANTHROPIC_BASE_URL=https://your-provider.com/v1
baoclaw

```

配置文件参考

详细的配置文件说明请参考英文版 [Configuration Reference](#) 部分。

简要概览：

文件	位置	说明
config.json	~/.baoclaw/	主配置（模型、API、Telegram token）
BAOCLAW.md	<项目>/~/.baoclaw/	项目指令，注入系统提示词
mcp.json	两级都有	MCP 服务器配置
mcp.local.json	<项目>/~/.baoclaw/	本地 MCP 覆盖（gitignore）
memory.jsonl	两级都有	记忆存储
cron.json	~/.baoclaw/	定时任务
skills/*.md	两级都有	技能文件
todo.json	<项目>/~/.baoclaw/	项目待办
evolution/	~/.baoclaw/	进化数据（轨迹、候选 skill）
sessions/	~/.baoclaw/	会话记录（按项目）

环境变量： - ANTHROPIC_API_KEY — API 密钥（必需） - ANTHROPIC_MODEL — 覆盖配置中的模型 - ANTHROPIC_BASE_URL — OpenAI 兼容 API 地址 - BRAVE_SEARCH_API_KEY — Web 搜索 API 密钥

完整命令列表

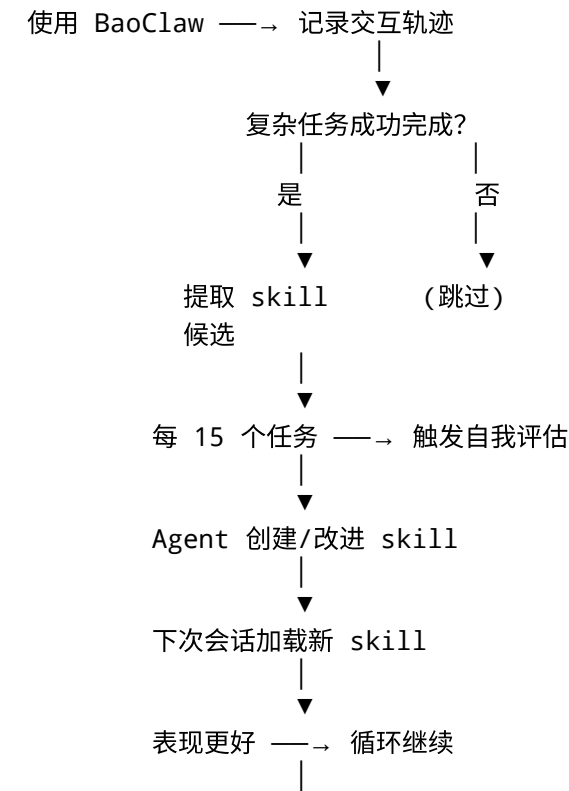
命令	说明
/projects	项目管理：list, , new <路径> [描述], desc
/tools	列出已注册的工具
/mcp	列出 MCP 服务器
/skills	列出已加载的技能
/plugins	列出已安装的插件
/model [名称]	查看或切换模型
/think	切换扩展思考模式
/compact	压缩对话上下文
/memory	长期记忆：list, add, delete, clear
/cron	定时任务：add, list, remove, toggle
/diff	查看 git diff
/commit <消息>	暂存并提交
/git	查看 git 状态

命令	说明
/task	后台任务：run, list, status, stop
/voice	语音输入（需要 whisper.cpp）
/telegram	管理 Telegram 网关
/telemetry	切换遥测
@file.pdf	附加文件进行问答
/abort	取消当前请求（或按 Ctrl+C）
/clear	清屏
/help	显示所有命令
/quit	断开连接（守护进程保持运行）
/shutdown	停止守护进程

定时任务示例

```
/cron add "每日git总结" "daily 09:00" 总结昨天的git提交
/cron add "依赖检查" "weekly mon 10:00" 检查项目依赖安全更新
/cron add "进化评估" "every 2h" 检查待处理的skill候选并改进
/cron list
/cron toggle abc123
/cron remove abc123
```

自我进化：工作原理





导出轨迹数据 —→ RLHF/DPO 微调小模型

许可证

MIT